

Unidad 5

Clases y objetos

Tabla de contenido

1. Introducción.....	3
2. Clases	3
2.1. Declaración de clase	4
2.2. Declaración de variables	5
2.3. Definición de métodos	7
2.4. Constructores	9
2.5. Pasar información a un método o constructor	10
3. Objetos	13
3.1. Creación de objetos	15
3.2. Uso de objetos	19
3.3. El recolector de basura	22
4. Paquetes.....	22
4.1. Creación de un paquete	23
4.2. Nombrar un paquete.....	24
4.3. Uso de miembros de paquetes	25
4.4. Gestión de archivos fuente y de clase.....	29
4.5. Resumen de la creación y el uso de paquetes	31
5. Más sobre clases	31
5.1. Devolución de valores por métodos	31
5.2. Uso de la palabra clave this	33
5.3. Control del acceso a los miembros de una clase.....	34
5.4. El modificador "static"	36
5.4.1. Variables de clase	36
5.4.2. Métodos de clase.....	38
5.4.3. Constantes.....	38
5.4.4. Las clases de Bicicleta	39
6. Interfaces.....	40
Ejercicios.....	44

1. Introducción.

Con el conocimiento que ahora tienes de los conceptos básicos del lenguaje de programación Java, puedes aprender a escribir tus propias clases. En esta lección, encontrarás información sobre cómo definir tus propias clases, incluida la declaración de atributos, métodos y constructores.

Aprenderás a usar tus clases para crear objetos y cómo usar los objetos que crees.

2. Clases

Este es un código de ejemplo para una posible implementación de una clase Bicycle. Las secciones siguientes de esta lección respaldarán y explicarán las declaraciones de clase paso a paso. Por el momento, no te preocupes por los detalles.

```
public class Bicycle {

    // the Bicycle class has
    // three fields
    public int cadence;
    public int gear;
    public int speed;

    // the Bicycle class has
    // one constructor
    public Bicycle(int startCadence, int startSpeed, int
                                                startGear) {
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;
    }

    // the Bicycle class has
    // four methods
    public void setCadence(int newValue) {
        cadence = newValue;
    }

    public void setGear(int newValue) {
        gear = newValue;
    }

    public void applyBrake(int decrement) {
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }
}
```

```
}
```

Una declaración de clase para una clase MountainBike que es una subclase de Bicycle podría tener el siguiente aspecto:

```
public class MountainBike extends Bicycle {

    // the MountainBike subclass has
    // one field
    public int seatHeight;

    // the MountainBike subclass has
    // one constructor
    public MountainBike(int startHeight, int startCadence,
                        int startSpeed, int startGear) {
        super(startCadence, startSpeed, startGear);
        seatHeight = startHeight;
    }

    // the MountainBike subclass has
    // one method
    public void setHeight(int newValue) {
        seatHeight = newValue;
    }

}
```

MountainBike hereda todos los atributos y métodos de Bicycle y agrega el atributo `seatHeight` y un método para establecerlo (las bicicletas de montaña tienen asientos que se pueden mover hacia arriba y hacia abajo según lo exija el terreno).

2.1. Declaración de clase

Has visto clases definidas de la siguiente manera:

```
class MyClass {
    // field, constructor, and
    // method declarations
}
```

Esta es una *declaración de clase*. El *cuerpo* de la clase (el área entre llaves) contiene todo el código que proporciona el ciclo de vida de los objetos creados a partir de la clase: constructores para inicializar nuevos objetos, declaraciones para los atributos que proporcionan el estado de la clase y sus objetos, y métodos para implementar el comportamiento de la clase y sus objetos.

La declaración de clase anterior es mínima. Contiene solo los componentes de una declaración de clase que son necesarios. Puede proporcionar más información sobre la clase, como el nombre de su superclase, si implementa interfaces, etc., al comienzo de la declaración de clase. Por ejemplo:

```
class MyClass extends MySuperClass implements YourInterface {  
    // field, constructor, and  
    // method declarations  
}
```

significa que `MyClass` es una subclase de `MySuperClass` y que implementa la interfaz `YourInterface`.

También puede agregar modificadores como *public* o *private* al principio. Puedes ver que la línea de apertura de una declaración de clase puede volverse bastante complicada. Los modificadores *public* y *private*, que determinan qué otras clases pueden tener acceso a `MyClass`, se describen más adelante en esta lección. La lección sobre interfaces y herencia explicará cómo y por qué usaría las *palabras clave extends e implements* en una declaración de clase. Por el momento, no necesitas preocuparse por estas complicaciones adicionales.

En general, las declaraciones de clase pueden incluir estos componentes, en orden:

1. Modificadores como *public*, *private* y varios otros que encontrarás más adelante.
2. El nombre de la clase, con la letra inicial en mayúsculas por convención.
3. El nombre la clase padre (superclase), si la hay, precedido por la palabra clave *extends*. Una clase solo puede *extender* (subclase) un padre.
4. Una lista separada por comas de interfaces implementadas por la clase, si las hay, precedida por la palabra clave *implements*. Una clase puede *implementar* más de una interfaz.
5. El cuerpo de la clase, rodeado por llaves, `{}`.

2.2. Declaración de variables

Hay varios tipos de variables:

- Variables miembro de una clase: se denominan ***atributos (fields en inglés)***.
- Variables en un método o bloque de código: se denominan ***variables locales***.
- Variables en declaraciones de métodos: se denominan ***parámetros***.

La clase `Bicycle` usa las siguientes líneas de código para definir sus atributos:

```
public int cadence;  
public int gear;  
public int speed;
```

Las declaraciones de atributos se componen de tres componentes, en orden:

1. Cero o más modificadores, como `public` o `private`.
2. El tipo de atributo.
3. El nombre del atributo.

Los atributos de Bicycle se denominan cadence, gear y speed y son todos de tipo de datos entero (int). La palabra clave public identifica estos atributos como miembros públicos, accesibles para cualquier objeto que pueda acceder a la clase.

Modificadores de acceso

El primer modificador (más a la izquierda) utilizado te permite controlar qué otras clases tienen acceso a un atributo miembro. Por el momento, considera solo public y private. Otros modificadores de acceso se discutirán más adelante.

- public (Modificador public): se puede acceder al atributo desde todas las clases.
- private : el atributo solo es accesible dentro de su propia clase.

En el espíritu de la encapsulación, es común hacer que los atributos sean privados. Esto significa que solo se puede *acceder directamente* desde la clase Bicycle. Sin embargo, todavía necesitamos acceder a estos valores. Esto se puede hacer *indirectamente* agregando métodos públicos que obtienen los valores del atributo para nosotros:

```
public class Bicycle {  
  
    private int cadence;  
    private int gear;  
    private int speed;  
  
    public Bicycle(int startCadence, int startSpeed,  
                  int startGear) {  
        gear = startGear;  
        cadence = startCadence;  
        speed = startSpeed;  
    }  
  
    public int getCadence() {  
        return cadence;  
    }  
  
    public void setCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    public int getGear() {  
        return gear;  
    }  
  
    public void setGear(int newValue) {  
        gear = newValue;  
    }  
  
    public int getSpeed() {  
        return speed;  
    }  
  
    public void applyBrake(int decrement) {
```

```
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }
}
```

Tipos

Todas las variables deben tener un tipo. Puedes usar tipos primitivos como `int`, `float`, `booleano`, etc. O bien, puedes usar tipos de referencia, como cadenas, matrices u objetos.

Nombres de variables

Todas las variables, ya sean atributos, variables locales o parámetros, siguen las mismas reglas y convenciones de nomenclatura que se trataron en la unidad 2 (Datos básicos para algoritmos y programas).

En esta lección, ten en cuenta que se usan las mismas reglas y convenciones de nomenclatura para los nombres de método y clase, excepto que

- la primera letra del nombre de una clase debe estar en mayúscula, y
- La primera (o única) palabra en el nombre de un método debe ser un **verbo**.

2.3. Definición de métodos

A continuación, se muestra un ejemplo de una declaración de método típica:

```
public double calculateAnswer(double wingSpan,
                             int numberOfEngines,
                             double length, double grossTons) {
    //do the calculation here
}
```

Los únicos elementos necesarios de una declaración de método son el tipo de valor devuelto del método, el nombre, un par de paréntesis, `()` y un cuerpo entre llaves, `{}`.

De manera más general, las declaraciones de método tienen seis componentes, en orden:

1. Modificadores, como `public`, `private` y otros de los que aprenderás más adelante.
2. El tipo de valor devuelto: el tipo de datos del valor devuelto por el método, o `void` si el método no devuelve un valor.
3. El nombre del método: las reglas para los nombres de atributo también se aplican a los nombres de método, pero la convención es un poco diferente.
4. La lista de parámetros entre paréntesis: una lista delimitada por comas de parámetros de entrada, precedida por sus tipos de datos, entre paréntesis, `()`. Si no hay parámetros, debes usar paréntesis vacíos.

5. Una lista de excepciones, que se discutirá más adelante.
6. El cuerpo del método, entre llaves, el código del método, incluida la declaración de variables locales, va aquí.

Los modificadores, los tipos de valor devuelto y los parámetros se tratarán más adelante en esta lección. Las excepciones se analizan en una lección posterior.

Definición: Dos de los componentes de una declaración de método comprenden la *signatura del método*: el nombre del método y los tipos de parámetros.

La signatura del método declarado anteriormente es:

```
calculateAnswer(double, int, double, double)
```

Nombrar un método

Aunque un nombre de método puede ser cualquier identificador legal, las convenciones de código restringen los nombres de método. Por convención, los nombres de los métodos deben ser un verbo en minúsculas o un nombre de varias palabras que comience con un verbo en minúsculas, seguido de adjetivos, sustantivos, etc. En los nombres de varias palabras, la primera letra de cada una de las segundas palabras y siguientes debe estar en mayúscula. Aquí hay algunos ejemplos:

```
run
runFast
getBackground
getFinalData
compareTo
setX
isEmpty
```

Un método puede tener el mismo nombre que otros métodos debido a la *sobrecarga de métodos*.

Sobrecarga de métodos

El lenguaje de programación Java admite sobrecarga de métodos, pudiendo distinguir entre métodos con diferentes *signaturas de método*. Esto significa que los métodos dentro de una clase pueden tener el mismo nombre si tienen diferentes listas de parámetros (hay algunas calificaciones para esto que se discutirán en la lección titulada "Interfaces y herencia").

Supongamos que tienes una clase que puede usar caligrafía para dibujar varios tipos de datos (cadenas, enteros, etc.) y que contiene un método para dibujar cada tipo de datos. Es engorroso usar un nuevo nombre para cada método, por ejemplo, drawString, drawInteger, drawFloat, etc. En el lenguaje de programación Java, puedes usar el mismo nombre para todos los métodos de dibujo, pero pasar una lista de argumentos diferente a cada método. Por lo tanto, la clase de dibujo de datos podría declarar cuatro métodos denominados draw, cada uno de los cuales tiene una lista de parámetros diferente.

```
public class DataArtist {
```



```
...
public void draw(String s) {
    ...
}
public void draw(int i) {
    ...
}
public void draw(double f) {
    ...
}
public void draw(int i, double f) {
    ...
}
}
```

Los métodos sobrecargados se diferencian por el número y/o el tipo de argumentos pasados al método. En el ejemplo de código, `draw(String s)` y `draw(int i)` son métodos distintos y únicos porque requieren diferentes tipos de argumentos.

No puede declarar más de un método con el mismo nombre y el mismo número y tipo de argumentos, porque el compilador no puede distinguirlos.

El compilador no tiene en cuenta el tipo de valor devuelto al diferenciar los métodos, por lo que no se pueden declarar dos métodos con la misma signatura, aunque tengan un tipo de valor devuelto diferente.

2.4. Constructores

Una clase contiene constructores que se invocan para crear objetos a partir del plano técnico de la clase. Las declaraciones de constructor se parecen a las declaraciones de método, excepto que usan el nombre de la clase y no tienen ningún tipo de valor devuelto. Por ejemplo, `Bicycle` tiene un constructor:

```
public Bicycle(int startCadence, int startSpeed, int startGear) {
    gear = startGear;
    cadence = startCadence;
    speed = startSpeed;
}
```

Para crear un nuevo objeto `Bicycle` llamado `myBike`, el operador `new` llama a un constructor:

```
Bicycle myBike = new Bicycle(30, 0, 8);
```

`new Bicycle(30, 0, 8)` crea espacio en la memoria para el objeto e inicializa sus atributos.

Aunque `Bicycle` solo tiene un constructor, podría tener otros, incluido un constructor sin argumentos:

```
public Bicycle() {
    gear = 1;
}
```

```
        cadence = 10;
        speed = 0;
    }
```

`Bicycle yourBike = new Bicycle();` invoca el constructor sin argumentos para crear un nuevo objeto `Bicycle` denominado `yourBike`.

Ambos constructores podrían haber sido declarados en `Bicycle` porque tienen diferentes listas de argumentos. Al igual que con los métodos, la plataforma Java diferencia a los constructores en función del número de argumentos en la lista y sus tipos. No puedes crear dos constructores que tengan el mismo número y tipo de argumentos para la misma clase, porque la plataforma no podría distinguirlos. Si lo haces, se produce un error en tiempo de compilación.

No es necesario que proporciones ningún constructor para la clase, pero debes tener cuidado al hacerlo. El compilador proporciona automáticamente un constructor predeterminado sin argumentos para cualquier clase sin constructores. Este constructor predeterminado llamará al constructor sin argumentos de la superclase. En esta situación, el compilador se quejará si la superclase no tiene un constructor sin argumentos, por lo que debes verificar que lo tenga. Si tu clase no tiene una superclase explícita, entonces tiene una superclase implícita de `Object`, que *tiene* un constructor sin argumento.

Puedes usar un constructor de superclase tú mismo. La clase `MountainBike` al comienzo de esta lección hizo precisamente eso. Esto se discutirá más adelante, en la lección sobre interfaces y herencia.

Puedes usar modificadores de acceso en la declaración de un constructor para controlar qué otras clases pueden llamar al constructor.

Nota: Si otra clase no puede llamar a un constructor `MyClass`, no puede crear directamente objetos `MyClass`.

2.5. Pasar información a un método o constructor

La declaración de un método o un constructor declara el número y el tipo de los parámetros para ese método o constructor. Por ejemplo, el siguiente es un método que calcula el precio neto dado el precio bruto (sin IVA), el porcentaje de IVA, el porcentaje de descuento:

```
public double calculateNetPrice(
    double grossPrice,
    double percentDiscount,
    int iva) {
    double discount = grossPrice * percentDiscount / 100.0;
    double tax = (grossPrice - discount) * iva / 100.0;
    double answer = grossPrice - discount + tax;

    return answer;
}
```

Este método tiene tres parámetros: el precio bruto, el descuento en % y el IVA. Los dos primeros son números en coma flotante de doble precisión y el tercero es un número entero. Los parámetros se usan en el cuerpo del método y en tiempo de ejecución tomarán los valores de los argumentos que se pasan.

Nota: *Los parámetros* se refieren a la lista de variables en una declaración de método. *Los argumentos* son los valores reales que se pasan cuando se invoca el método. Cuando se invoca un método, los argumentos utilizados deben coincidir con los parámetros de la declaración en tipo y orden.

Tipos de parámetros

Puedes usar cualquier tipo de datos para un parámetro de un método o un constructor. Esto incluye tipos de datos primitivos, como dobles, flotantes y enteros, como se vio en el método `calculateNetPrice`, y tipos de datos de referencia, como objetos y matrices.

Este es un ejemplo de un método que acepta un array como argumento. En este ejemplo, el método crea un nuevo objeto `Polygon` y lo inicializa a partir de un array de objetos `Point` (supongamos que `Point` es una clase que representa una coordenada `x, y`):

```
public Polygon polygonFrom(Point[] corners) {  
    // method body goes here  
}
```

Nota: El lenguaje de programación Java no permite pasar métodos a métodos. Pero puede pasar un objeto a un método y, a continuación, invocar los métodos del objeto.

Número arbitrario de argumentos

Puede usar una construcción llamada *varargs* para pasar un número arbitrario de valores a un método. Se usa *varargs* cuando no se sabe cuántos de un tipo particular de argumento se pasarán al método. Es un atajo para crear un array manualmente (el método anterior podría haber usado *varargs* en lugar de un array).

Para usar *varargs*, añada puntos suspensivos (tres puntos, `...`), al tipo del último parámetro, luego un espacio y el nombre del parámetro. A continuación, se puede llamar al método con cualquier número de ese parámetro, incluido ninguno.

```
public Polygon polygonFrom(Point... corners) {  
    int numberOfPoints = corners.length;  
    double squareOfSide1, lengthOfSide1;  
    squareOfSide1 = (corners[1].x - corners[0].x)  
        * (corners[1].x - corners[0].x)  
        + (corners[1].y - corners[0].y)  
        * (corners[1].y - corners[0].y);  
    lengthOfSide1 = Math.sqrt(squareOfSide1);  
  
    // more method body code follows that creates and returns a  
    // polygon connecting the Points  
}
```

Puedes ver que, dentro del método, `corners` se trata como un array. El método se puede llamar con un array o con una secuencia de argumentos separados por comas. El código en el cuerpo del método tratará el parámetro como un array, en cualquier caso.

Nombres de parámetros

Cuando se declara un parámetro en un método o un constructor, se proporciona un nombre para ese parámetro. Este nombre se usa dentro del cuerpo del método para hacer referencia al argumento pasado.

El nombre de un parámetro debe ser único en su ámbito. No puede ser el mismo que el nombre de otro parámetro para el mismo método o constructor, y no puede ser el nombre de una variable local dentro del método o constructor.

Un parámetro puede tener el mismo nombre que uno de los atributos de la clase. Si este es el caso, se dice que el parámetro *apantalla* el atributo. Los atributos apantallados pueden dificultar la lectura del código y solo se usan convencionalmente dentro de constructores y métodos que establecen un atributo determinado. Por ejemplo, considera la siguiente clase `Circle` y su método `setOrigin`:

```
public class Circle {
    private int x, y, radius;
    public void setOrigin(int x, int y) {
        ...
    }
}
```

La clase `Circle` tiene tres atributos: `x`, `y` y `radius`. El método `setOrigin` tiene dos parámetros, cada uno de los cuales tiene el mismo nombre que uno de los atributos. Cada parámetro de método apantalla al atributo que comparte su nombre. Por lo tanto, el uso de los nombres simples `x` o `y` dentro del cuerpo del método se refieren al parámetro, *no* al atributo. Para acceder al atributo, debes usar un nombre completo. Esto se discutirá más adelante en esta lección en la sección titulada "Uso de la palabra clave `this`".

Pasar argumentos de tipo de datos primitivos

Los argumentos primitivos, como un `int` o un `double`, se pasan a los métodos *por valor*. Esto significa que cualquier cambio en los valores de los parámetros solo existe dentro del alcance del método. Cuando el método vuelve, los parámetros desaparecen y se pierden los cambios en ellos. Aquí hay un ejemplo:

```
public class PassPrimitiveByValue {

    public static void main(String[] args) {

        int x = 3;

        // invoke passMethod() with
        // x as argument
        passMethod(x);
    }
}
```

```
// print x to see if its
// value has changed
System.out.println("After invoking passMethod, x = " + x);

}

// change parameter in passMethod()
public static void passMethod(int p) {
    p = 10;
}
}
```

Cuando ejecutas este programa, el resultado es:

```
After invoking passMethod, x = 3
```

Pasar argumentos de tipo de datos de referencia

Los parámetros de tipo de datos de referencia, como los objetos, también se pasan a los métodos *por valor*. Esto significa que cuando el método devuelve, la referencia pasada sigue haciendo referencia al mismo objeto que antes. *Sin embargo*, los valores de los atributos del objeto se *pueden* cambiar en el método, si tienen el nivel de acceso adecuado.

Por ejemplo, considera un método en una clase arbitraria que mueve objetos Circle:

```
public void moveCircle(Circle circle, int deltaX, int deltaY) {
    // code to move origin of circle to x+deltaX, y+deltaY
    circle.setX(circle.getX() + deltaX);
    circle.setY(circle.getY() + deltaY);

    // code to assign a new reference to circle
    circle = new Circle(0, 0);
}
```

Sea el método invocado con estos argumentos:

```
moveCircle(myCircle, 23, 56)
```

Dentro del método, circle inicialmente hace referencia a myCircle. El método cambia las coordenadas x e y del objeto al que hace referencia el círculo (es decir, myCircle) en 23 y 56, respectivamente. Estos cambios persistirán cuando se devuelva el método. A continuación, a circle se le asigna una referencia a un nuevo objeto Circle con x = y = 0. Sin embargo, esta reasignación no tiene permanencia porque la referencia se pasó por valor y no puede cambiar. Dentro del método, el objeto al que apunta circle ha cambiado, pero, cuando el método devuelve, myCircle sigue haciendo referencia al mismo objeto Circle que antes de llamar al método.

3. Objetos

Para ilustrar esta sección usaremos tres clases:

```
public class Point {
    public int x = 0;
    public int y = 0;
    // a constructor!
    public Point(int a, int b) {
        x = a;
        y = b;
    }
}

public class Rectangle {
    public int width = 0;
    public int height = 0;
    public Point origin;

    // four constructors
    public Rectangle() {
        origin = new Point(0, 0);
    }
    public Rectangle(Point p) {
        origin = p;
    }
    public Rectangle(int w, int h) {
        origin = new Point(0, 0);
        width = w;
        height = h;
    }
    public Rectangle(Point p, int w, int h) {
        origin = p;
        width = w;
        height = h;
    }

    // a method for moving the rectangle
    public void move(int x, int y) {
        origin.x = x;
        origin.y = y;
    }

    // a method for computing the area of the rectangle
    public int getArea() {
        return width * height;
    }
}
```

Y la clase principal:

```
public class CreateObjectDemo {

    public static void main(String[] args) {
        Point originOne = new Point(23, 94);
        Rectangle rectOne = new Rectangle(originOne, 100, 200);
    }
}
```

```
Rectangle rectTwo = new Rectangle(50, 100);

// display rectOne's width, height, and area
System.out.println("Width of rectOne: " + rectOne.width);
System.out.println("Height of rectOne: " +
    rectOne.height);
System.out.println("Area of rectOne: " +
    rectOne.getArea());

// set rectTwo's position
rectTwo.origin = originOne;

// display rectTwo's position
System.out.println("X Position of rectTwo: " +
    rectTwo.origin.x);
System.out.println("Y Position of rectTwo: " +
    rectTwo.origin.y);

// move rectTwo and display its new position
rectTwo.move(40, 72);
System.out.println("X Position of rectTwo: " +
    rectTwo.origin.x);
System.out.println("Y Position of rectTwo: " +
    rectTwo.origin.y);
    }
}
```

Este programa crea, manipula y muestra información sobre varios objetos. Aquí está el resultado:

```
Width of rectOne: 100
Height of rectOne: 200
Area of rectOne: 20000
X Position of rectTwo: 23
Y Position of rectTwo: 94
X Position of rectTwo: 40
Y Position of rectTwo: 72
```

3.1. Creación de objetos

Como sabes, una clase proporciona el plano técnico para los objetos; creas un objeto a partir de una clase. Cada una de las instrucciones siguientes tomadas del programa `CreateObjectDemo` crea un objeto y lo asigna a una variable:

```
Point originOne = new Point(23, 94);
Rectangle rectOne = new Rectangle(originOne, 100, 200);
Rectangle rectTwo = new Rectangle(50, 100);
```

La primera línea crea un objeto de la clase `Point` y la segunda y la tercera línea crean cada una un objeto de la clase `Rectangle`.

Cada una de estas declaraciones tiene tres partes (que se analizan en detalle a continuación):

1. **Declaración:** el código establecido en **negrita** son todas las declaraciones de variables que asocian un nombre de variable con un tipo de objeto.
2. **Creación de instancias:** la palabra clave `new` es un operador de Java que crea el objeto.
3. **Inicialización:** el operador `new` va seguido de una llamada a un constructor, que inicializa el nuevo objeto.

Declarar una variable para hacer referencia a un objeto

Anteriormente, aprendiste que, para declarar una variable, escribes:

```
type name;
```

Esto notifica al compilador que usará *name* para hacer referencia a los datos cuyo tipo es *type*. Con una variable primitiva, esta declaración también reserva la cantidad adecuada de memoria para la variable.

También puedes declarar una variable de referencia en su propia línea. Por ejemplo:

```
Point originOne;
```

Si declaras `originOne` de esta manera, su valor será indeterminado hasta que se cree y se le asigne un objeto. Simplemente declarar una variable de referencia no crea un objeto. Para ello, debes usar el operador `new`, como se describe en la siguiente sección. Debes asignar un objeto a `originOne` antes de usarlo en el código. De lo contrario, obtendrás un error del compilador.

Una variable en este estado, que actualmente no hace referencia a ningún objeto, se puede ilustrar de la siguiente manera (el nombre de la variable, `originOne`, más una referencia que no apunta a nada):

`originOne` 

Creación de instancias de una clase

El operador `new` crea una instancia de una clase asignando memoria para un nuevo objeto y devolviendo una referencia a esa memoria. El operador `new` también invoca el constructor de objetos.

Nota: La frase "instanciar una clase" significa lo mismo que "crear un objeto". Cuando crea un objeto, está creando una "instancia" de una clase, por lo tanto, "instancia" una clase.

El operador `new` requiere un único argumento de sufijo: una llamada a un constructor. El nombre del constructor proporciona el nombre de la clase de la que se va a crear una instancia.

El operador `new` devuelve una referencia al objeto que creó. Esta referencia generalmente se asigna a una variable del tipo apropiado, como:

```
Point originOne = new Point(23, 94);
```

La referencia devuelta por el `nuevo` operador no tiene que asignarse a una variable. También se puede usar directamente en una expresión. Por ejemplo:

```
int height = new Rectangle().height;
```

Esta declaración se discutirá en la siguiente sección.

Inicialización de un objeto

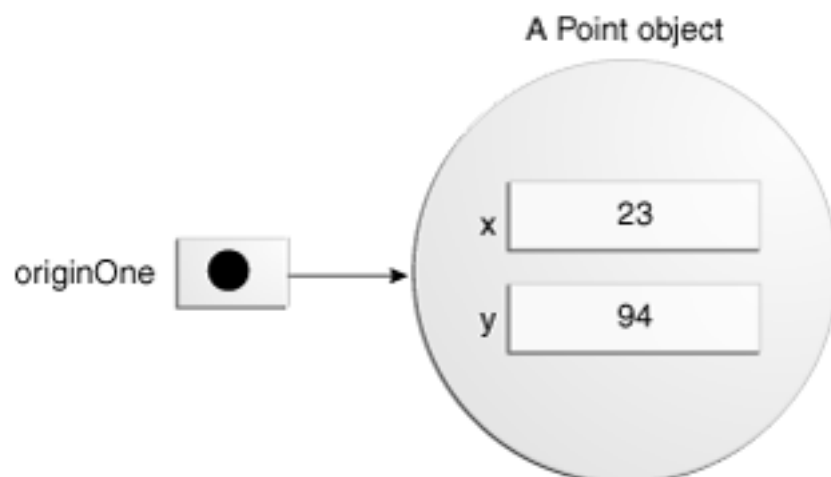
Aquí está el código para la clase `Point`:

```
public class Point {  
    public int x = 0;  
    public int y = 0;  
    //constructor  
    public Point(int a, int b) {  
        x = a;  
        y = b;  
    }  
}
```

Esta clase contiene un único constructor. Puedes reconocer que es un constructor porque su declaración usa el mismo nombre que la clase y no tiene ningún tipo de valor devuelto. El constructor de la clase `Point` toma dos argumentos enteros, según lo declarado por el código (`int a, int b`). La siguiente instrucción proporciona 23 y 94 como valores para esos argumentos:

```
Point originOne = new Point(23, 94);
```

El resultado de la ejecución de esta instrucción se puede ilustrar en la siguiente figura:



Este es el código de la clase `Rectangle`, que contiene cuatro constructores:

```
public class Rectangle {
    public int width = 0;
    public int height = 0;
    public Point origin;

    // four constructors
    public Rectangle() {
        origin = new Point(0, 0);
    }
    public Rectangle(Point p) {
        origin = p;
    }
    public Rectangle(int w, int h) {
        origin = new Point(0, 0);
        width = w;
        height = h;
    }
    public Rectangle(Point p, int w, int h) {
        origin = p;
        width = w;
        height = h;
    }

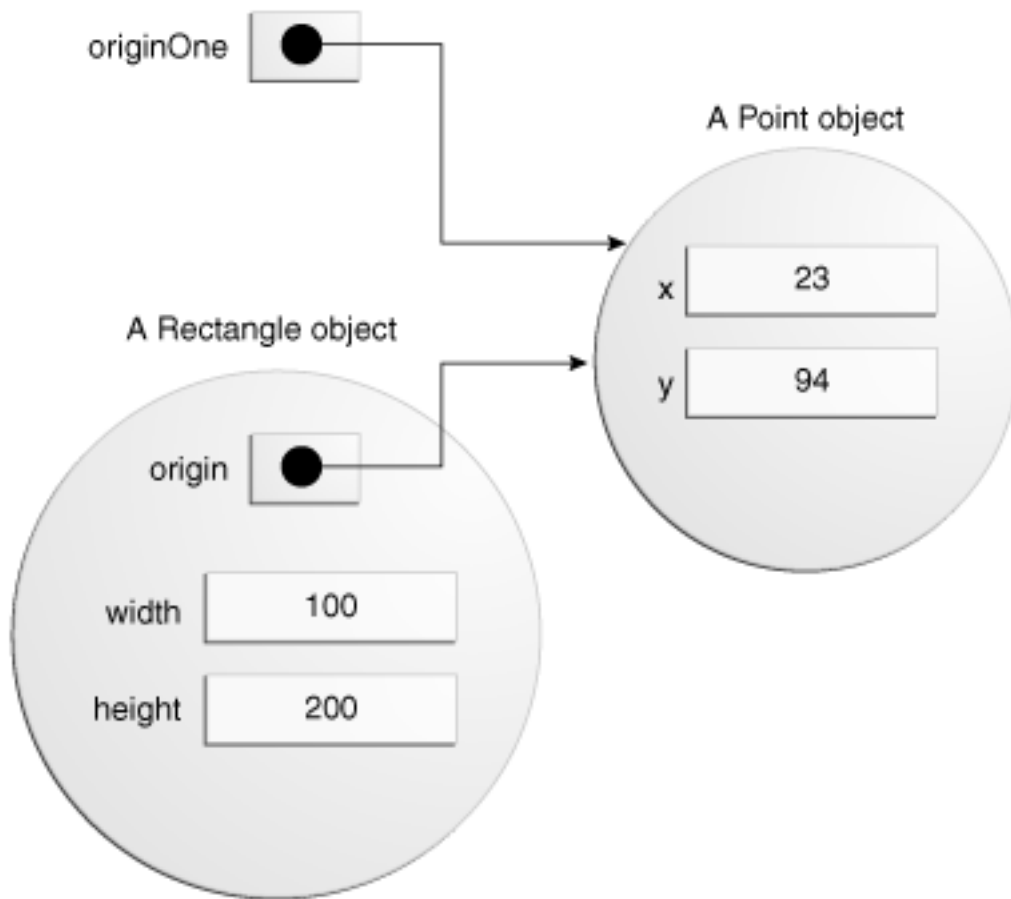
    // a method for moving the rectangle
    public void move(int x, int y) {
        origin.x = x;
        origin.y = y;
    }

    // a method for computing the area of the rectangle
    public int getArea() {
        return width * height;
    }
}
```

Cada constructor permite proporcionar valores iniciales para el tamaño y el ancho del rectángulo, utilizando tipos primitivos y de referencia. Si una clase tiene varios constructores, deben tener signatures diferentes. El compilador de Java diferencia los constructores en función del número y el tipo de argumentos. Cuando el compilador de Java encuentra el siguiente código, sabe que debe llamar al constructor de la clase `Rectangle` que requiere un argumento `Point` seguido de dos argumentos enteros:

```
Rectangle rectOne = new Rectangle(originOne, 100, 200);
```

Esto llama a uno de los constructores de `Rectangle` que inicializa `origin` a `originOne`. Además, el constructor establece el `width` en 100 y el `height` en 200. Ahora hay dos referencias al mismo objeto `Point`: un objeto puede tener varias referencias a él, como se muestra en la siguiente figura:



La siguiente línea de código llama al constructor `Rectangle` que requiere dos argumentos enteros, que proporcionan los valores iniciales de `width` y `height`. Si inspeccionas el código dentro del constructor, verás que crea un nuevo objeto `Point` cuyos valores `x` e `y` se inicializan en 0:

```
Rectangle rectTwo = new Rectangle(50, 100);
```

El constructor `Rectangle` que se usa en la siguiente instrucción no toma ningún argumento, por lo que se denomina *constructor sin argumentos*:

```
Rectangle rect = new Rectangle();
```

Todas las clases tienen al menos un constructor. Si una clase no declara explícitamente ninguna, el compilador de Java proporciona automáticamente un constructor sin argumentos, denominado *constructor predeterminado*. Este constructor predeterminado llama al constructor sin argumento del elemento primario de la clase, o al constructor `Object` si la clase no tiene otro elemento primario. Si el padre no tiene constructor (`Object` tiene uno), el compilador rechazará el programa.

3.2. Uso de objetos

Una vez que hayas creado un objeto, probablemente desees usarlo para algo. Es posible que tengas que usar el valor de uno de sus atributos, cambiar uno de sus atributos o llamar a uno de sus métodos para realizar una acción.

Se accede a los atributos de objeto por su nombre. Debes usar un nombre que no sea ambiguo.

Puedes usar un nombre simple para un atributo dentro de su propia clase. Por ejemplo, podemos agregar una declaración *dentro* de la clase `Rectangle` que imprime el ancho y el alto:

```
System.out.println("Width and height are: " + width + ", " + height);
```

En este caso, el ancho y el alto son nombres simples.

El código que está fuera de la clase del objeto debe usar una referencia o expresión de objeto, seguida del operador de punto (`.`), seguido de un nombre de atributo simple, como en:

```
objectReference.fieldName
```

Por ejemplo, el código de la clase `CreateObjectDemo` está fuera del código de la clase `Rectangle`. Por lo tanto, para hacer referencia a los atributos de origen, ancho y alto dentro del objeto `Rectangle` denominado `rectOne`, la clase `CreateObjectDemo` debe usar los nombres `rectOne.origin`, `rectOne.width` y `rectOne.height`, respectivamente. El programa usa dos de estos nombres para mostrar el ancho y el alto de `rectOne`:

```
System.out.println("Width of rectOne: " + rectOne.width);  
System.out.println("Height of rectOne: " + rectOne.height);
```

Intentar usar los nombres simples `width` y `height` del código en la clase `CreateObjectDemo` no tiene sentido, esos atributos existen solo dentro de un objeto, y da como resultado un error del compilador.

Más tarde, el programa usa un código similar para mostrar información sobre `rectTwo`. Los objetos del mismo tipo tienen su propia copia de los mismos atributos de instancia. Por lo tanto, cada objeto `Rectangle` tiene atributos denominados `origin`, `width` y `height`. Cuando se accede a un atributo de instancia a través de una referencia de objeto, se hace referencia al atributo de ese objeto en particular. Los dos objetos `rectOne` y `rectTwo` del programa `CreateObjectDemo` tienen atributos de `origin`, `width` y `height` diferentes .

Para acceder a un atributo, puedes usar una referencia con nombre a un objeto, como en los ejemplos anteriores, o puedes usar cualquier expresión que devuelva una referencia de objeto. Recuerda que el operador `new` devuelve una referencia a un objeto. Por lo tanto, puedes usar el valor devuelto por `new` para acceder a los atributos de un nuevo objeto:

```
int height = new Rectangle().height;
```

Esta instrucción crea un nuevo objeto `Rectangle` e inmediatamente obtiene su `height`. En esencia, la instrucción calcula la altura predeterminada de un rectángulo. Ten en cuenta que después de que se haya ejecutado esta instrucción, el programa ya no tiene

una referencia al `Rectangle` creado, porque el programa nunca almacenó la referencia en ningún lugar. El objeto no está referenciado y sus recursos pueden ser reciclados libremente por la máquina virtual Java.

Llamar a los métodos de un objeto

También se usa una referencia de objeto para invocar el método de un objeto. Anexe el nombre simple del método a la referencia del objeto, con un operador de punto intermedio (`.`). Además, proporciona, entre paréntesis, cualquier argumento al método. Si el método no requiere ningún argumento, usa paréntesis vacíos.

```
objectReference.methodName(argumentList);
```

o:

```
objectReference.methodName();
```

La clase `Rectangle` tiene dos métodos: `getArea()` para calcular el área del rectángulo y `move()` para cambiar el origen del rectángulo. Este es el código `CreateObjectDemo` que invoca estos dos métodos:

```
System.out.println("Area of rectOne: " + rectOne.getArea());
...

rectTwo.move(40, 72);
```

La primera instrucción invoca el método `getArea()` de `rectOne` y muestra los resultados. La segunda línea mueve `rectTwo` porque el método `move()` asigna nuevos valores a `origin.x` y `origin.y` del objeto.

Al igual que con los atributos de instancia, *objectReference* debe ser una referencia a un objeto. Puedes usar un nombre de variable, pero también puedes usar cualquier expresión que devuelva una referencia de objeto. El operador `new` devuelve una referencia de objeto, por lo que puedes usar el valor devuelto de `new` para invocar los métodos de un nuevo objeto:

```
new Rectangle(100, 50).getArea()
```

La expresión `new Rectangle(100, 50)` devuelve una referencia de objeto que hace referencia a un objeto `Rectangle`. Como se muestra, puedes usar la notación de puntos para invocar el método `getArea()` del nuevo `Rectangle` para calcular el área del nuevo rectángulo.

Algunos métodos, como `getArea()`, devuelven un valor. Para los métodos que devuelven un valor, puede usar la invocación de método en expresiones. Puede asignar el valor devuelto a una variable, usarlo para tomar decisiones o controlar un bucle. Este código asigna el valor devuelto por `getArea()` a la variable `areaOfRectangle`:

```
int areaOfRectangle = new Rectangle(100, 50).getArea();
```

Recuerda, invocar un método en un objeto determinado es lo mismo que enviar un mensaje a ese objeto. En este caso, el objeto en el que se invoca `getArea()` es el rectángulo devuelto por el constructor.

3.3. El recolector de basura

Algunos lenguajes orientados a objetos requieren que se realice un seguimiento de todos los objetos que cree y que los destruya explícitamente cuando ya no sean necesarios. Administrar la memoria explícitamente es tedioso y propenso a errores. La plataforma Java te permite crear tantos objetos como quieras (limitados, por supuesto, por lo que tu sistema puede manejar), y no tienes que preocuparte por destruirlos. El entorno de ejecución de Java suprime los objetos cuando determina que ya no se utilizan. Este proceso se denomina *recolección de elementos no utilizados*.

Un objeto es apto para la recolección de elementos no utilizados cuando no hay más referencias a ese objeto. Las referencias que se mantienen en una variable generalmente se descartan cuando la variable sale del ámbito. O bien, puede quitar explícitamente una referencia de objeto estableciendo la variable en el valor especial null. Recuerde que un programa puede tener múltiples referencias al mismo objeto; Todas las referencias a un objeto deben quitarse antes de que el objeto sea elegible para la recolección de elementos no utilizados.

El entorno de ejecución de Java tiene un recolector de elementos no utilizados que libera periódicamente la memoria utilizada por los objetos a los que ya no se hace referencia. El recolector de elementos no utilizados hace su trabajo automáticamente cuando determina que es el momento adecuado.

4. Paquetes

Un paquete es un espacio de nombres que organiza un conjunto de clases e interfaces relacionadas. Conceptualmente, puedes pensar en los paquetes como similares a diferentes carpetas en tu computadora. Puedes mantener las páginas HTML en una carpeta, las imágenes en otra y los scripts o aplicaciones en otra. Debido a que el software escrito en el lenguaje de programación Java puede estar compuesto por cientos o miles de clases individuales, tiene sentido mantener las cosas organizadas colocando clases e interfaces relacionadas en paquetes.

Los nombres de los paquetes están organizados como nombres de dominio inversos como: `com.mypackages.graphics`, por ejemplo.

La plataforma Java proporciona una enorme biblioteca de clases (un conjunto de paquetes) adecuada para su uso en sus propias aplicaciones. Esta biblioteca se conoce como "Interfaz de programación de aplicaciones" o "API" para abreviar. Sus paquetes representan las tareas más comúnmente asociadas con la programación de propósito general. Por ejemplo, un objeto String contiene el estado y el comportamiento de las cadenas de caracteres; un objeto File permite a un programador crear, eliminar, inspeccionar, comparar o modificar fácilmente un archivo en el sistema de archivos; un objeto Socket permite la creación y el uso de sockets de red; varios objetos GUI, botones de control y casillas de verificación y cualquier otra cosa relacionada con las interfaces gráficas de usuario. Hay literalmente miles de clases para elegir. Esto permite al

programador, concentrarse en el diseño de su aplicación en particular, en lugar de la infraestructura necesaria para que funcione.

La [especificación de API de la plataforma Java](#) contiene la lista completa de todos los paquetes, interfaces, clases, atributos y métodos proporcionados por la plataforma Java SE. Carga la página en tu navegador y márcala. Como programador, se convertirá en tu documentación de referencia más importante.

4.1. Creación de un paquete

Para crear un paquete, elige un nombre para el paquete (las convenciones de nomenclatura se describen en la sección siguiente) y coloca una instrucción `package` con ese nombre en la parte superior de *cada archivo fuente* que contenga los tipos (clases, interfaces, enumeraciones y anotaciones) que deseas incluir en el paquete.

La instrucción `package` (por ejemplo, `package graphics;`) debe ser la primera línea del archivo fuente. Solo puede haber una instrucción `package` en cada archivo de código fuente y se aplica a todos los tipos del archivo.

Nota: Si colocas varios tipos en un solo archivo de origen, solo uno puede ser público y debe tener el mismo nombre que el archivo fuente. Por ejemplo, puedes definir la `public class Circle` en el fichero `Circle.java`, definir `public interface Draggable` en el fichero `Draggable.java`, definir `public enum Day` en el fichero `Day.java`, etc.

Puedes incluir tipos no públicos en el mismo archivo que un tipo público (esto se desaconseja encarecidamente, a menos que los tipos no públicos sean pequeños y estén estrechamente relacionados con el tipo público), pero solo se podrá acceder al tipo público desde fuera del paquete. Todos los tipos no públicos de nivel superior serán *package private*.

Si colocas la interfaz gráfica y las clases enumeradas en la sección anterior en un paquete llamado `graphics`, necesitarías seis archivos fuente, como este:

```
//in the Draggable.java file
package graphics;
public interface Draggable {
    . . .
}

//in the Graphic.java file
package graphics;
public abstract class Graphic {
    . . .
}

//in the Circle.java file
package graphics;
public class Circle extends Graphic
    implements Draggable {
    . . .
}
```

```
//in the Rectangle.java file
package graphics;
public class Rectangle extends Graphic
    implements Draggable {
    . . .
}

//in the Point.java file
package graphics;
public class Point extends Graphic
    implements Draggable {
    . . .
}

//in the Line.java file
package graphics;
public class Line extends Graphic
    implements Draggable {
    . . .
}
```

Si no usas una instrucción `package`, el tipo termina en un paquete sin nombre. En términos generales, un paquete sin nombre es solo para aplicaciones pequeñas o temporales o cuando recién está comenzando el proceso de desarrollo. De lo contrario, las clases y las interfaces pertenecen a paquetes con nombre.

4.2. Nombrar un paquete

Con programadores de todo el mundo escribiendo clases e interfaces utilizando el lenguaje de programación Java, es probable que muchos programadores usen el mismo nombre para diferentes tipos. De hecho, el ejemplo anterior hace precisamente eso: define una clase `Rectangle` cuando ya hay una clase `Rectangle` en el paquete `java.awt`. Aún así, el compilador permite que ambas clases tengan el mismo nombre si están en paquetes diferentes. El nombre completo de cada clase `Rectangle` incluye el nombre del paquete. Es decir, el nombre completo de la clase `Rectangle` en el paquete de gráficos es `graphics.Rectangle` y el nombre completo de la clase `Rectangle` en el paquete `java.awt` es `java.awt.Rectangle`.

Esto funciona bien a menos que dos programadores independientes usen el mismo nombre para sus paquetes. ¿Qué previene este problema? Convención.

Convenciones de nomenclatura

Los nombres de los paquetes se escriben en minúsculas para evitar conflictos con los nombres de las clases o interfaces.

Las empresas usan su nombre de dominio de Internet invertido para comenzar sus nombres de paquetes, por ejemplo, `com.example.mypackage` para un paquete llamado `mypackage` creado por un programador en `example.com`.

Las colisiones de nombres que se producen dentro de una sola empresa deben controlarse por convención dentro de esa empresa, tal vez incluyendo la región o el nombre del proyecto después del nombre de la empresa (por ejemplo, `com.example.region.mypackage`).

Los paquetes en el propio lenguaje Java comienzan con `java.` o `Javax.`

En algunos casos, el nombre de dominio de Internet puede no ser un nombre de paquete válido. Esto puede ocurrir si el nombre de dominio contiene un guión u otro carácter especial, si el nombre del paquete comienza con un dígito u otro carácter que no se puede usar como comienzo de un nombre de Java, o si el nombre del paquete contiene una palabra clave de Java reservada, como `"int"`. En este caso, la convención sugerida es agregar un guión bajo. Por ejemplo:

Legalización de nombres de paquetes

Nombre de dominio	Prefijo de nombre de paquete
<code>hyphenated-name.example.org</code>	<code>org.example.hyphenated_name</code>
<code>example.int</code>	<code>int_.example</code>
<code>123name.example.com</code>	<code>com.example._123name</code>

4.3. Uso de miembros de paquetes

Los tipos que componen un paquete se conocen como miembros de *paquete*.

Para usar un miembro de paquete `public` desde fuera de su paquete, debes realizar una de las siguientes acciones:

- Hacer referencia al miembro por su nombre completo
- Importar el miembro del paquete
- Importar el paquete completo del miembro

Cada uno es apropiado para diferentes situaciones, como se explica en las secciones siguientes.

Hacer referencia a un miembro de paquete por su nombre completo

Hasta ahora, la mayoría de los ejemplos de este tutorial se han referido a los tipos por sus nombres simples, como `Rectangle` y `StackOfInts`. Puedes usar el nombre simple de un miembro del paquete si el código que estás escribiendo está en el mismo paquete que ese miembro o si ese miembro se ha importado.

Sin embargo, si estás intentando usar un miembro de un paquete diferente y ese paquete no se ha importado, debes usar el nombre completo del miembro, que incluye el nombre del paquete. Este es el nombre completo de la clase `Rectangle` declarada en el paquete de gráficos del ejemplo anterior.

```
graphics.Rectangle
```

Puedes usar este nombre completo para crear una instancia de `graphics.Rectangle`:

```
graphics.Rectangle myRect = new graphics.Rectangle();
```

Los nombres calificados están bien para un uso poco frecuente. Sin embargo, cuando un nombre se usa repetidamente, escribir el nombre repetidamente se vuelve tedioso y el código se vuelve difícil de leer. Como alternativa, puedes *importar* el miembro o su paquete y, a continuación, usar su nombre simple.

Importar un miembro del paquete

Para importar un miembro específico en el archivo actual, coloca una instrucción `import` al principio del archivo antes de las definiciones de tipo, pero después de la instrucción `package`, si la hay. Así es como importaría la clase `Rectangle` desde el paquete de gráficos creado en la sección anterior.

```
import graphics.Rectangle;
```

Ahora puedes hacer referencia a la clase `Rectangle` por su nombre simple.

```
Rectangle myRectangle = new Rectangle();
```

Este enfoque funciona bien si usas solo unos pocos miembros del paquete de gráficos. Pero si usas muchos tipos de un paquete, debes importar el paquete completo.

Importación de un paquete completo

Para importar todos los tipos contenidos en un paquete determinado, utiliza la instrucción `import` con el carácter comodín de asterisco (*).

```
import graphics.*;
```

Ahora puedes hacer referencia a cualquier clase o interfaz en el paquete de gráficos por su nombre simple.

```
Circle myCircle = new Circle();  
Rectangle myRectangle = new Rectangle();
```

El asterisco en la instrucción `import` solo se puede usar para especificar todas las clases dentro de un paquete, como se muestra aquí. No se puede usar para hacer coincidir un subconjunto de las clases de un paquete. Por ejemplo, lo siguiente no coincide con todas las clases del paquete de gráficos que comienzan por `A`.

```
// does not work  
import graphics.A*;
```

En su lugar, genera un error del compilador. Con la instrucción `import`, generalmente se importa solo un miembro de paquete o un paquete completo.

Nota: Otra forma de importación menos común permite importar las clases públicas anidadas de una clase envolvente. Por ejemplo, si los `graphics.Rectangle` contenía clases anidadas útiles, como `Rectangle.DoubleWide` y `Rectangle.Square`, podrías importar `Rectangle` y sus clases anidadas mediante las *dos* instrucciones siguientes.

```
import graphics.Rectangle;
import graphics.Rectangle.*;
```

Ten en cuenta que la segunda instrucción de importación *no* importará `Rectangle`.

Otra forma menos común de importación, la *instrucción de importación estática*, se discutirá al final de esta sección.

Para mayor comodidad, el compilador de Java importa automáticamente dos paquetes completos para cada archivo de origen: (1) el paquete `java.lang` y (2) el paquete actual (el paquete para el archivo actual).

Jerarquías aparentes de paquetes

Al principio, los paquetes parecen ser jerárquicos, pero no lo son. Por ejemplo, la API de Java incluye un paquete `java.awt`, un paquete `java.awt.color`, un paquete `java.awt.font` y muchos otros que comienzan con `java.awt`. Sin embargo, el paquete `java.awt.color`, el paquete `java.awt.font` y otros paquetes `java.awt.xxxx` *no se incluyen* en el paquete `java.awt`. El prefijo `java.awt` (Java Abstract Window Toolkit) se utiliza para una serie de paquetes relacionados para hacer evidente la relación, pero no para mostrar la inclusión.

La importación de `java.awt.*` importa todos los tipos del paquete `java.awt`, pero *no importa* `java.awt.color`, `java.awt.font` ni ningún otro paquete `java.awt.xxxx`. Si planeas usar las clases y otros tipos en `java.awt.color`, así como los de `java.awt`, debes importar ambos paquetes con todos sus archivos:

```
import java.awt.*;
import java.awt.color.*;
```

Ambigüedades de nombres

Si un miembro de un paquete comparte su nombre con un miembro de otro paquete y ambos paquetes se importan, debes hacer referencia a cada miembro por su nombre completo. Por ejemplo, el paquete de gráficos definió una clase denominada `Rectangle`. El paquete `java.awt` también contiene una clase `Rectangle`. Si se han importado gráficos y `java.awt`, lo siguiente es ambiguo.

```
Rectangle rect;
```

En tal situación, debe usar el nombre completo del miembro para indicar exactamente qué clase `Rectangle` desea. Por ejemplo

```
graphics.Rectangle rect;
```

La instrucción de importación estática

Hay situaciones en las que se necesita acceso frecuente a atributos finales estáticos (constantes) y métodos estáticos de una o dos clases. Prefijar el nombre de estas clases una y otra vez puede dar como resultado un código desordenado. La instrucción *static import* proporciona una manera de importar las constantes y los métodos estáticos que desees usar para que no se necesite prefijar el nombre de su clase.

La clase `java.lang.Math` define la constante `PI` y muchos métodos estáticos, incluidos métodos para calcular senos, cosenos, tangentes, raíces cuadradas, máximos, mínimos, exponentes y muchos más. Por ejemplo

```
public static final double PI
    = 3.141592653589793;
public static double cos(double a)
{
    ...
}
```

Normalmente, para usar estos objetos de otra clase, se antepone el nombre de la clase, como se indica a continuación.

```
double r = Math.cos(Math.PI * theta);
```

Puedes usar la instrucción *static import* para importar los miembros estáticos de `java.lang.Math` para que no se necesite prefijar el nombre de clase, `Math`. Los miembros estáticos de `Math` se pueden importar individualmente:

```
import static java.lang.Math.PI;
```

o en grupo:

```
import static java.lang.Math.*;
```

Una vez importados, los miembros estáticos se pueden utilizar sin calificación. Por ejemplo, el fragmento de código anterior se convertiría en:

```
double r = cos(PI * theta);
```

Obviamente, puedes escribir tus propias clases que contengan constantes y métodos estáticos que uses con frecuencia y, a continuación, usar la instrucción *static import*. Por ejemplo:

```
import static mypackage.MyConstants.*;
```

Nota: Utiliza la importación estática con mucha moderación. El uso excesivo de la importación estática puede dar lugar a un código difícil de leer y mantener, ya que los lectores del código no sabrán qué clase define un objeto estático determinado. Si se usa correctamente, la importación estática hace que el código sea más legible al eliminar la repetición del nombre de clase.

4.4. Gestión de archivos fuente y de clase

Muchas implementaciones de la plataforma Java se basan en sistemas de archivos jerárquicos para administrar archivos de código fuente y de clase, aunque *la especificación del lenguaje Java* no lo requiere. La estrategia es la siguiente.

Coloca el código fuente de una clase, interfaz, enumeración o tipo de anotación en un archivo de texto cuyo nombre sea el nombre simple del tipo y cuya extensión sea .java. Por ejemplo:

```
//in the Rectangle.java file
package graphics;
public class Rectangle {
    ...
}
```

Luego, coloca el archivo fuente en un directorio cuyo nombre refleje el nombre del paquete al que pertenece el tipo:

```
.....\graphics\Rectangle.java
```

El nombre completo del miembro del paquete y el nombre de la ruta de acceso al archivo son paralelos, suponiendo que la barra diagonal inversa del separador de nombres de archivo de Microsoft Windows (para Unix, use la barra diagonal).

- **Nombre de la clase** : graphics.Rectangle
- **Nombre de ruta al archivo** – graphics\Rectangle.java

Como debes recordar, por convención, una empresa utiliza su nombre de dominio de Internet invertido para sus nombres de paquete. La empresa Example, cuyo nombre de dominio de Internet es example.com, precedería a todos sus nombres de paquete con com.example. Cada componente del nombre del paquete corresponde a un subdirectorio. Por lo tanto, si la empresa Example tuviera un paquete com.example.graphics que contuviera un archivo fuente Rectangle.java, estaría contenido en una serie de subdirectorios como este:

```
....\com\example\graphics\Rectangle.java
```

Al compilar un archivo de código fuente, el compilador crea un archivo de salida diferente para cada tipo definido en él. El nombre base del archivo de salida es el nombre del tipo y su extensión es .class. Por ejemplo, si el archivo de origen es así

```
//in the Rectangle.java file
package com.example.graphics;
public class Rectangle {
    ...
}

class Helper{
```

```
    . . .  
}
```

A continuación, los archivos compilados se ubicarán en:

```
<path to the parent directory of the output  
files>\com\example\graphics\Rectangle.class  
<path to the parent directory of the output  
files>\com\example\graphics\Helper.class
```

Al igual que los archivos de código fuente .java, los archivos .class compilados deben estar en una serie de directorios que reflejen el nombre del paquete. Sin embargo, la ruta de acceso a los archivos .class no tiene que ser la misma que la ruta de acceso a los archivos de código fuente .java. Puede organizar los directorios de origen y clase por separado, como:

```
<path_one>\sources\com\example\graphics\Rectangle.java  
  
<path_two>\classes\com\example\graphics\Rectangle.class
```

Al hacer esto, puedes dar el directorio de clases a otros programadores sin revelar sus fuentes. También debes administrar los archivos de código fuente y de clase de esta manera para que el compilador y la máquina virtual Java (JVM) puedan encontrar todos los tipos que usa tu programa.

La ruta completa al directorio de clases, <path_two>\classes, se denomina ruta de *clase* y se establece con la variable de sistema CLASSPATH. Tanto el compilador como la JVM construyen la ruta a los archivos .class agregando el nombre del paquete a la ruta de la clase. Por ejemplo, si

```
<path_two>\classes
```

es la ruta de su clase y el nombre del paquete es

```
com.example.graphics,
```

luego el compilador y la JVM buscan archivos .class en

```
<path_two>\classes\com\example\graphics.
```

Una ruta de clase puede incluir varias rutas, separadas por un punto y coma (Windows) o dos puntos (Unix). De forma predeterminada, el compilador y la JVM buscan en el directorio actual y en el archivo JAR que contiene las clases de la plataforma Java para que estos directorios estén automáticamente en la vía de acceso de la clase.

Establecer la variable de sistema CLASSPATH

Para mostrar la variable CLASSPATH actual, usa estos comandos en Windows y Unix (shell Bourne):

```
En Windows:    C:\> set CLASSPATH
```

```
En Unix:      % echo $CLASSPATH
```

Para establecer la variable CLASSPATH, usa estos comandos (por ejemplo):

```
En Windows:   C:\> set CLASSPATH=C:\users\george\java\classes
En Unix:      % CLASSPATH=/home/george/java/classes
              % export CLASSPATH
```

Para eliminar el contenido actual de la variable CLASSPATH, utiliza estos comandos:

```
En Windows:   C:\> set CLASSPATH=
En Unix:      % unset CLASSPATH
              % export CLASSPATH
```

4.5. Resumen de la creación y el uso de paquetes

Para crear un paquete para un tipo, coloque una instrucción `package` como la primera instrucción en el archivo de código fuente que contiene el tipo (clase, interfaz, enumeración o tipo de anotación).

Para usar un tipo público que está en un paquete diferente, tienes tres opciones: (1) usar el nombre completo del tipo, (2) importar el tipo o (3) importar el paquete completo del que el tipo es miembro.

Los nombres de ruta de acceso de los archivos de origen y clase de un paquete reflejan el nombre del paquete.

Es posible que tengas que establecer tu CLASSPATH para que el compilador y la JVM puedan encontrar los archivos `.class` para tus tipos.

5. Más sobre clases

En esta sección se tratan más aspectos de las clases que dependen del uso de referencias a objetos y el operador de punto que aprendió en las secciones anteriores sobre objetos:

- Devolución de valores por métodos.
- La palabra clave `this`.
- Miembros de clase frente a instancia.
- Control de acceso.

5.1. Devolución de valores por métodos

Un método vuelve al código que lo invocó cuando

- completa todas las sentencias del método,
- llega a una declaración de `return`, o

- lanza una excepción (que se trata más adelante),

lo que ocurra primero.

El tipo de valor devuelto de un método se declara en su signature de método. Dentro del cuerpo del método, se usa la instrucción `return` para devolver el valor.

Cualquier método declarado `void` no devuelve un valor. No es necesario que contenga una declaración `return`, pero puede hacerlo. En tal caso, se puede usar una instrucción `return` para ramificar un bloque de flujo de control y salir del método y simplemente se usa así:

```
return;
```

Si intentas devolver un valor de un método que se declara `void`, obtendrás un error del compilador.

Cualquier método que no se declare `void` debe contener una declaración `return` con un valor de retorno correspondiente, como este:

```
return returnValue;
```

El tipo de datos del valor devuelto debe coincidir con el tipo de valor devuelto declarado del método; No puede devolver un valor entero de un método declarado para devolver un valor booleano.

El método `getArea()` de la clase `Rectangle` que se discutió en las secciones sobre objetos devuelve un número entero:

```
// a method for computing the area of the rectangle
public int getArea() {
    return width * height;
}
```

Este método devuelve el entero al que se evalúa la expresión `width*height`.

El método `getArea` devuelve un tipo primitivo. Un método también puede devolver un tipo de referencia. Por ejemplo, en un programa para manipular objetos `Bicycle`, podríamos tener un método como este:

```
public Bicycle seeWhosFastest(Bicycle myBike, Bicycle yourBike,
                             Environment env) {
    Bicycle fastest;
    // code to calculate which bike is
    // faster, given each bike's gear
    // and cadence and given the
    // environment (terrain and wind)
    return fastest;
}
```


5.2. Uso de la palabra clave `this`

Dentro de un método de instancia o un constructor, `this` es una referencia al *objeto actual*, el objeto cuyo método o constructor se está llamando. Puedes hacer referencia a cualquier miembro del objeto actual desde un método de instancia o un constructor mediante `this`.

Usar `this` con un atributo

La razón más común para usar la palabra clave `this` es porque un atributo está apantallado por un método o parámetro del constructor.

Por ejemplo, la clase `Point` se escribió así

```
public class Point {
    public int x = 0;
    public int y = 0;

    //constructor
    public Point(int a, int b) {
        x = a;
        y = b;
    }
}
```

pero podría haberse escrito así:

```
public class Point {
    public int x = 0;
    public int y = 0;

    //constructor
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

Cada argumento del constructor apantalla uno de los atributos del objeto: dentro del constructor `x` hay una copia local del primer argumento del constructor. Para hacer referencia al atributo `Point x`, el constructor debe usar `this.x`.

Usando `this` con un constructor.

Desde dentro de un constructor, también puedes usar la palabra clave **this** para llamar a otro constructor en la misma clase. Al hacerlo, se denomina *invocación explícita del constructor*. Aquí hay otra clase `Rectangle`, con una implementación diferente a la de la sección Objetos.

```
public class Rectangle {
    private int x, y;
    private int width, height;

    public Rectangle() {
        this(0, 0, 0, 0);
    }
    public Rectangle(int width, int height) {
        this(0, 0, width, height);
    }
    public Rectangle(int x, int y, int width, int height) {
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
    }
    ...
}
```

Esta clase contiene un conjunto de constructores. Cada constructor inicializa algunas o todas las variables miembro del rectángulo. Los constructores proporcionan un valor predeterminado para cualquier variable miembro cuyo valor inicial no sea proporcionado por un argumento. Por ejemplo, el constructor sin argumentos llama al constructor de cuatro argumentos con cuatro valores 0 y el constructor de dos argumentos llama al constructor de cuatro argumentos con dos valores 0. Como antes, el compilador determina a qué constructor llamar, según el número y el tipo de argumentos.

Si está presente, la invocación de otro constructor debe ser la primera línea del constructor.

5.3. Control del acceso a los miembros de una clase

Los modificadores de nivel de acceso determinan si otras clases pueden usar un atributo determinado o invocar un método determinado. Hay dos niveles de control de acceso:

- En el nivel superior (aplicado las clases): `public` o *package-private* (sin modificador explícito).
- En el nivel de miembro (aplicado a Atributos y Métodos): `public`, `private`, `protected` o *package-private* (sin modificador explícito).

Una clase se puede declarar con el modificador `public`, en cuyo caso esa clase es visible para todas las clases en todas partes. Si una clase no tiene modificador (el valor predeterminado, también conocido como *package-private*), solo es visible dentro de su propio paquete (los paquetes son grupos con nombre de clases relacionadas; aprenderás sobre ellos en una lección posterior).

En el nivel de miembro, también puede usar el modificador `public` o no modificador (*package-private*) al igual que con las clases de nivel superior, y con el mismo significado. Para los miembros, hay dos modificadores de acceso adicionales: `private` y `protected`. El modificador `private` especifica que solo se puede tener acceso al miembro en su propia clase. El modificador `protected` especifica que solo se puede

tener acceso al miembro dentro de su propio paquete (como con *package-private*) y, además, mediante una subclase de su clase en otro paquete.

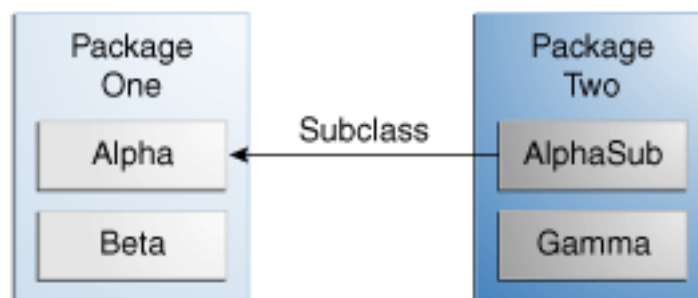
En la tabla siguiente se muestra el acceso a los miembros permitido por cada modificador.

Niveles de acceso				
Modificador	Clase	Paquete	Subclase	Mundo
public	Y	Y	Y	Y
protected	Y	Y	Y	N
<i>sin modificador</i>	Y	Y	N	N
private	Y	N	N	N

La primera columna de datos indica si la propia clase tiene acceso al miembro definido por el nivel de acceso. Como puede ver, una clase siempre tiene acceso a sus propios miembros. La segunda columna indica si las clases del mismo paquete que la clase (independientemente de su parentesco) tienen acceso al miembro. La tercera columna indica si las subclases de la clase declarada fuera de este paquete tienen acceso al miembro. La cuarta columna indica si todas las clases tienen acceso al miembro.

Los niveles de acceso te afectan de dos maneras. En primer lugar, cuando se utilizan clases que proceden de otro origen, como las clases de la plataforma Java, los niveles de acceso determinan qué miembros de esas clases pueden utilizar las propias clases. En segundo lugar, cuando escribe una clase, debe decidir qué nivel de acceso debe tener cada variable miembro y cada método de su clase.

Veamos una colección de clases y veamos cómo los niveles de acceso afectan la visibilidad. En la ilustración siguiente se muestran las cuatro clases de este ejemplo y cómo se relacionan.



Clases y paquetes del ejemplo utilizado para ilustrar los niveles de acceso

En la tabla siguiente se muestra dónde están visibles los miembros de la clase Alpha para cada uno de los modificadores de acceso que se les pueden aplicar.

Modificador	Visibilidad			
	Alfa	Beta	Alphasub	Gamma
<code>public</code>	Y	Y	Y	Y
<code>protected</code>	Y	Y	Y	N
<i>sin modificador</i>	Y	Y	N	N
<code>private</code>	Y	N	N	N

Consejos para elegir un nivel de acceso:

Si otros programadores usan su clase, desea asegurarse de que no se produzcan errores por mal uso. Los niveles de acceso pueden ayudar a hacer esto.

- Usa el nivel de acceso más restrictivo que tenga sentido para un miembro en particular. Úsa `private` a menos que tengas una buena razón para no hacerlo.
- Evita los atributos `public` excepto para las constantes. (Muchos de los ejemplos del tutorial usan atributos públicos. Esto puede ayudar a ilustrar algunos puntos de manera concisa, pero no se recomienda para el código de producción). Los atributos públicos tienden a vincularlo a una implementación en particular y limitan su flexibilidad para cambiar su código.

5.4. El modificador "static"

Usamos la palabra clave **static** para crear atributos y métodos que pertenecen a la clase, en lugar de a una instancia de la clase.

5.4.1. Variables de clase

Cuando se crean varios objetos a partir del mismo plano técnico de clase, cada uno tiene sus propias copias distintas de variables de *instancia*. En el caso de la clase `Bicycle`, las variables de instancia son `cadence`, `gear` y `speed`. Cada objeto `Bicycle` tiene sus propios valores para estas variables, almacenados en diferentes ubicaciones de memoria.

A veces, desea tener variables que sean comunes a todos los objetos. Esto se logra con el modificador `static`. Los atributos que tienen el modificador `static` en su declaración se denominan **atributos estáticos** o **variables de clase**. Están asociados con la clase, más que con cualquier objeto. Cada instancia de la clase comparte una variable de clase, que se encuentra en una ubicación fija en la memoria. Cualquier objeto puede cambiar el valor de una variable de clase, pero las variables de clase también se pueden manipular sin crear una instancia de la clase.

Por ejemplo, supongamos que deseas crear una serie de objetos `Bicycle` y asignar a cada uno un número de serie, comenzando por 1 para el primer objeto. Este número de

ID es único para cada objeto y, por lo tanto, es una variable de instancia. Al mismo tiempo, necesita un atributo para realizar un seguimiento de cuántos objetos Bicycle se han creado para saber qué ID asignar al siguiente. Tal atributo no está relacionado con ningún objeto individual, sino con la clase en su conjunto. Para ello necesitas una variable de clase, `numberOfBicycles`, como se indica a continuación:

```
public class Bicycle {  
  
    private int cadence;  
    private int gear;  
    private int speed;  
  
    // add an instance variable for the object ID  
    private int id;  
  
    // add a class variable for the  
    // number of Bicycle objects instantiated  
    private static int numberOfBicycles = 0;  
    ...  
}
```

Las variables de clase son referenciadas por el propio nombre de clase, como en

```
Bicycle.numberOfBicycles
```

Esto deja en claro que son variables de clase.

Nota: También puedes hacer referencia a atributos estáticos con una referencia de objeto como:

```
myBike.numberOfBicycles
```

pero esto se desaconseja porque no deja claro que son variables de clase.

Puedes usar el constructor `Bicycle` para establecer la variable de instancia `id` e incrementar la variable de clase `numberOfBicycles`:

```
public class Bicycle {  
  
    private int cadence;  
    private int gear;  
    private int speed;  
    private int id;  
    private static int numberOfBicycles = 0;  
  
    public Bicycle(int startCadence, int startSpeed,  
                   int startGear){  
        gear = startGear;  
        cadence = startCadence;  
        speed = startSpeed;  
  
        // increment number of Bicycles  
        // and assign ID number  
        id = ++numberOfBicycles;  
    }  
}
```

```
    }

    // new method to return the ID instance variable
    public int getID() {
        return id;
    }

    ...
}
```

5.4.2. Métodos de clase

El lenguaje de programación Java admite métodos estáticos y variables estáticas. Los métodos estáticos, que tienen el modificador `static` en sus declaraciones, deben invocarse con el nombre de la clase, sin necesidad de crear una instancia de la clase, como en

```
ClassName.methodName(args)
```

Nota: También puedes hacer referencia a métodos estáticos con una referencia de objeto como

```
instanceName.methodName(args)
```

pero esto se desaconseja porque no deja claro que son métodos de clase.

Un uso común de los métodos estáticos es acceder a atributos estáticos. Por ejemplo, podríamos agregar un método estático a la clase `Bicycle` para acceder al atributo estático `numberOfBicycles`:

```
public static int getNumberOfBicycles() {
    return numberOfBicycles;
}
```

No se permiten todas las combinaciones de variables y métodos de instancia y clase:

- Los métodos de instancia pueden acceder directamente a las variables de instancia y a los métodos de instancia.
- Los métodos de instancia pueden acceder directamente a las variables de clase y a los métodos de clase.
- Los métodos de clase pueden acceder directamente a las variables de clase y a los métodos de clase.
- Los métodos de clase **no pueden** acceder directamente a las variables de instancia o a los métodos de instancia, deben usar una referencia de objeto. Además, los métodos de clase no pueden usar la palabra clave `this` ya que no hay ninguna instancia a la que hacer referencia a `this`.

5.4.3. Constantes

El modificador `static`, en combinación con el modificador `final`, también se usa para definir constantes. El modificador `final` indica que el valor de este atributo no puede cambiar.

Por ejemplo, la siguiente declaración de variable define una constante denominada `PI`, cuyo valor es una aproximación de pi (la relación entre la circunferencia de un círculo y su diámetro):

```
static final double PI = 3.141592653589793;
```

Las constantes definidas de esta manera no se pueden reasignar y es un error en tiempo de compilación si el programa intenta hacerlo. Por convención, los nombres de los valores constantes se escriben en mayúsculas. Si el nombre se compone de más de una palabra, las palabras se separan con un carácter de subrayado (`_`).

Nota: Si un tipo primitivo o una cadena se define como una constante y el valor se conoce en tiempo de compilación, el compilador reemplaza el nombre de la constante en todas partes del código con su valor. Esto se llama *constante de tiempo de compilación*. Si el valor de la constante en el mundo exterior cambia (por ejemplo, si se legisla que pi en realidad debería ser 3.975), deberá volver a compilar las clases que usan esta constante para obtener el valor actual.

5.4.4. Las clases de Bicicleta

Después de todas las modificaciones realizadas en este apartado, la clase `Bicycle` es ahora:

```
public class Bicycle {

    private int cadence;
    private int gear;
    private int speed;

    private int id;

    private static int numberOfBicycles = 0;

    public Bicycle(int startCadence,
                   int startSpeed,
                   int startGear){
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;

        id = ++numberOfBicycles;
    }

    public int getID() {
        return id;
    }
}
```

```
    }

    public static int getNumberOfBicycles() {
        return numberOfBicycles;
    }

    public int getCadence(){
        return cadence;
    }

    public void setCadence(int newValue){
        cadence = newValue;
    }

    public int getGear(){
        return gear;
    }

    public void setGear(int newValue){
        gear = newValue;
    }

    public int getSpeed(){
        return speed;
    }

    public void applyBrake(int decrement){
        speed -= decrement;
    }

    public void speedUp(int increment){
        speed += increment;
    }
}
```

6. Interfaces

A veces puede ser interesante pedirle a un programador que desarrolle una clase que tenga ciertos métodos para trabajar en conjunto con otras clases. Imagina que estás creando un software que controla vehículos que pueden hablar. Puede pedirles a todos los fabricantes que construyen vehículos que trabajen con su sistema que implementen el método `talk()`.

Los programadores pueden cumplir con estas especificaciones y afirmar que cumplirán con estos requisitos.

La implementación de una interfaz permite que una clase se vuelva más formal sobre el comportamiento que promete proporcionar. Las interfaces forman un contrato entre la clase y el mundo exterior, y el compilador aplica este contrato en tiempo de compilación. Si su clase afirma implementar una interfaz, todos los métodos definidos por esa interfaz deben aparecer en su código fuente antes de que la clase se compile correctamente, de lo contrario se generará un error del compilador.

La estructura de una interfaz es similar a la de las clases. En lugar de la clase de palabras reservada **class**, usa **interface**. Y luego se declaran los encabezados de los métodos.

```
interface Talking {  
  
    // Headers of the methods that the  
    // interface force to implement go here  
    void talk();  
  
}
```

Pej.:

```
class TalkingBicycle extends Bicycle implements Talking {  
  
    // This class is forced to implement the talk() method  
  
    public void talk() {  
        System.out.println("I'm the incredible talking bike");  
    }  
  
    // Methods in the interface must be public.  
  
}
```

Otro punto importante sobre las interfaces es que Java no permite la herencia de más de una clase. La implementación de interfaces se puede utilizar como alternativa, lo que puede compensar esta carencia.

Ejemplo. Dos clases que implementan la interfaz de `Talking`:

```
interface Talking {
    void talk();
}

class Person implements Talking {
    String DNI;
    public void talk() {
        System.out.println("I am a person");
    }
}

class Bicycle implements Talking {
    int cadence = 0;
    int speed = 0;
    int gear = 1;

    void changeCadence(int newValue) {
        cadence = newValue;
    }

    void changeGear(int newValue) {
        gear = newValue;
    }

    void speedUp(int increment) {
        speed = speed + increment;
    }

    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println("cadence:" +
            cadence + " speed:" +
            speed + " gear:" + gear);
    }
    public void talk() {
        System.out.println("I'm the incredible talking bike");
    }
}
```

```
class UsingInterfaces {  
  
    public static void makeItTalk(Talking t) {  
        t.talk();  
    }  
  
    public static void main(String argv[]) {  
        Bicycle bike = new Bicycle();  
        Person person = new Person();  
        makeItTalk(bike);  
        makeItTalk(person);  
    }  
}
```

Ten en cuenta que en la clase principal **UsingInterfaces** declaramos un método **makeItTalk** que acepta un parámetro del tipo **Talking**. Esto significa que el parámetro puede ser de cualquier clase siempre que implemente la interfaz **Talking**. Esto es importante porque en el método **makeItTalk** llamamos al método **talk()** del parámetro.

Ejercicios

1. Escribir un programa para controlar las llamadas en una centralita telefónica.

En la centralita se registran las llamadas. Es posible que desee utilizar un array para almacenar todas las llamadas. Todas las llamadas tienen el número de origen de la llamada, el número de destino y la duración en segundos.

La centralita tendrá un método `print()` que imprime todas las llamadas, y otro `print()` que imprimirá los datos de la llamada dándole el orden de la llamada.

Hay dos tipos de llamadas:

- Las llamadas locales cuestan 15 centavos por segundo.
- Y llamadas provinciales en función del tiempo en el que costó la realización : 20 céntimos en el tiempo 1 , tiempo 2 25 céntimos y 30 céntimos en el tiempo 3, cada segundo.

Desarrolle una clase llamada `Exercise5`. En su método principal, cree una centralita, registre varios tipos diferentes y llamadas a la centralita e imprima el informe del número total de llamadas y la facturación total.

Sugerencia: Crea la clase `Call`.

2. Escribe un programa llamado `WordGuess` (y una clase `MagicWord`) para adivinar una palabra tratando de adivinar los caracteres individuales. La palabra a adivinar se proporcionará aleatoriamente a partir de una serie de palabras disponibles. Tu programa se verá así:

```
Enter one character or your guess word: t
Attempt 1: t__t__
Enter one character or your guess word: g
Attempt 2: t__t__g
Enter one character or your guess word: e
Attempt 3: te_t__g
Enter one character or your guess word: testing
Attempt 4: Congratulation!
          You got in 4 attempts
```

Consejos:

- Crea un array de `boolean` para indicar las posiciones de la palabra que se han adivinado correctamente.
- Comprueba la longitud de la entrada `String` para determinar si el jugador ingresa un solo carácter o una palabra adivinada. Si el jugador ingresa un solo carácter, compáralo con la palabra que se va a adivinar y actualiza el al array de `boolean` que mantiene el resultado hasta ahora.

3. **(De la unidad anterior, pero ahora con una clase).** Escribe un programa que simule el juego Battle-ship sobre un tablero de mesa de 8x8. La computadora colocará 10 naves aleatorias en él (solo 1 celda de longitud).

El usuario ingresará una coordenada (primero letra y luego número) y el programa escribirá el tablero completo cada vez con los barcos hundidos (X) y los disparos fallidos (círculos). Mostrará el contador de disparos y el número de barcos hundidos

Ejemplo:

SHOTS: 12

SUNK SHIPS: 3

	1	2	3	4	5	6	7	8
A								
B		O		O		O		
C		O					O	
D	X	O						
E				X	O		O	
F	O			O				
G	O				O		X	
H								O

Enter row (Letter): C

Enter column (Number): 2

4. Ejercicio: The Author and Book Classes

Author
<code>-name:String -email:String -gender:char</code>
<code>+Author(name:String, email:String, gender:char) +getName():String +getEmail():String +setEmail(email:String):void +getGender():char +toString():String</code>

Una clase denominada `Author` está diseñada como se muestra en el diagrama de clases. Contiene:

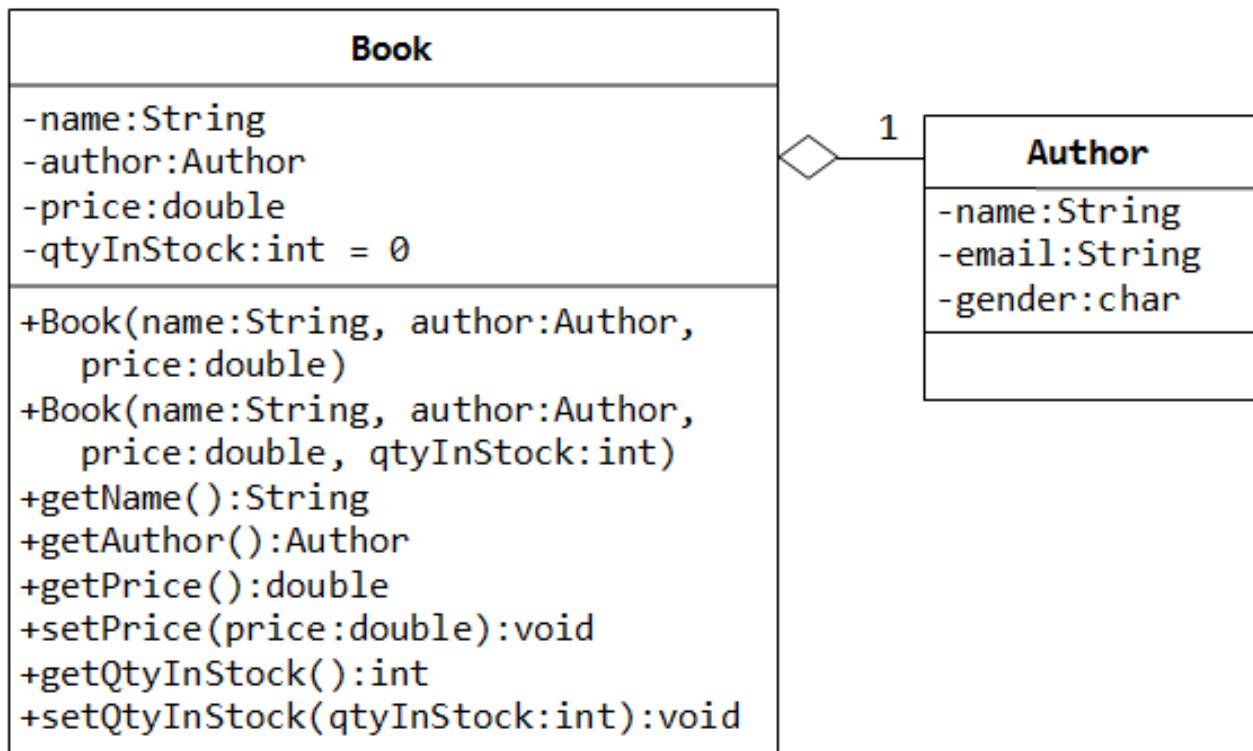
- Tres variables de instancia privadas: `name (String)`, `email (String)`, y `gender (char o bien 'm' o 'f')`;
 - Un constructor para inicializar el `name`, `email` y `gender` con los valores dados;

```
public Author (String name, String email, char gender)
{.....}
```

(No hay un constructor predeterminado para `Author`, ya que no hay valores predeterminados para el nombre, el correo electrónico y el género).
 - public **getters/setters**: `getName()`, `getEmail()`, `setEmail()`, y `getGender()`;
- (No hay `setters` para `name` y `gender`, ya que estos atributos no se pueden cambiar).
- Un `toString()` método que devuelve "author-name (gender) at email", e.g., "Tan Ah Teck (m) at ahTeck@somewhere.com".

Escribe la clase `Author`. Escribe también un programa de prueba denominado `TestAuthor` para probar el constructor y los métodos públicos. Intenta cambiar el `email` de un autor, por ejemplo:

```
Author anAuthor = new Author("Tan Ah Teck",
    "ahteck@somewhere.com", 'm');
System.out.println(anAuthor);    // call toString()
anAuthor.setEmail("paul@nowhere.com")
System.out.println(anAuthor);
```



Una clase llamada `Book` está diseñada como se muestra en el diagrama de clases. Contiene:

- Cuatro Variables de instancia privadas: `name` (`String`), `author` (de la clase `Author` que acabas de crear, supongamos que cada libro tiene un y solo un autor), `price` (`double`), y `qtyInStock` (`int`);
- Dos constructores: `public Book (String name, Author author, double price) {...}`
- `public Book (String name, Author author, double price,`
- `int qtyInStock) {...}`
- Métodos públicos `getName()`, `getAuthor()`, `getPrice()`, `setPrice()`, `getQtyInStock()`, `setQtyInStock()`.
- `toString()` que devuelve `"book-name' by author-name (gender) at email"`. (Ten en cuenta que el método `toString()` de `Author` devuelve `"author-name (gender) at email"`.)

Escribe la clase `Book` (que usa la clase `Author` escrita anteriormente). También escribe un programa de prueba llamado `TestBook` para probar el constructor y los métodos públicos en la clase `Book`. Ten en cuenta que debes construir una instancia de `Author` antes de poder construir una instancia de `Book`. Por ejemplo:

```

Author anAuthor = new Author(.....);
Book aBook = new Book("Java for dummy", anAuthor, 19.95,
1000);
// Use an anonymous instance of Author
Book anotherBook = new Book("more Java for dummy", new
Author(.....), 29.95, 888);
  
```

Ten en cuenta que las clases `Book` y `Author` tienen una variable llamada `name`. Sin embargo, se puede diferenciar a través de la instancia. Para una instancia de `Book` como `aBook`, `aBook.name` se refiere al nombre del libro; mientras que para una instancia de `Author`, como `anAuthor`, `anAuthor.name` se refiere al nombre del autor. No es necesario (y no se recomienda) llamar a las variables `bookName` y `authorName`.

PROBAR:

1. Imprimir el archivo nombre y Correo electrónico del autor de un Libro instancia. (Pista: `aLibro.getAutor().getName()`, `aLibro.getAutor().getEmail()`).
2. Introducir nuevos métodos llamados `getAuthorName()`, `getAuthorEmail()`, `getAuthorGender()` En `Book` para devolver el atributo `name`, `email` y `gender` del autor del libro. Por ejemplo `public String getAuthorName() { ..... }`
3. Crea un array de 5 libros. Inicialízalos con algunos datos y luego imprímelos todos.
4. Diseña una clase llamada `Triangle`, que modela un triángulo con 3 vértices. Se diseñará de la siguiente manera. La clase `MyTriangle` usa tres instancias de `Point` (creadas en un ejercicio anterior) como los tres vértices.

La clase contiene:

- Tres variables de instancia privadas `v1`, `v2`, `v3` (instancias de `Point`), para los tres vértices.
- Un constructor que construye un `Triangle` con tres puntos: `x1`, `y1`, `x2`, `y2`, `x3`, `y3`.
- Un constructor sobrecargado que construye un `Triangle` dados tres instancias de `Point`.
- Un `toString()` que devuelve una descripción de tipo `string` de la instancia en el formato `"Triangle @ (x1, y1), (x2, y2), (x3, y3)"`.
- Un `getPerimeter()` que devuelve la longitud del perímetro como `double`.
- Creación de un método estático en la clase `Point` `distance()` que acepta dos puntos como parámetro y devuelve la distancia entre ellos.
- Un método `printType()`, que imprime "equilátero" si los tres lados son iguales, "isósceles" si dos de los tres lados son iguales o "escaleno" si los tres lados son diferentes.

Escribe la clase `Triangle`. También escribe un programa de prueba (llamado `TestMyTriangle`) para probar todos los métodos definidos en la clase.

5. Diseña una clase llamada `MyPolynomial`, que modela polinomios de grado- n (ver ecuación), está diseñada como se muestra en el diagrama de clases.

$$c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$$

La clase contiene:

- Una variable de instancia denominada `coeffs`, que almacena los coeficientes del polinomio de grado n en un array de `double` de tamaño $n+1$, donde `c0` se mantiene en el índice 0.
- Un constructor `MyPolynomial(coeffs:double...)` Toma un número variable de `doubles` para inicializar el array `coeffs`, donde el primer argumento corresponde a `C0`. Los tres puntos se conocen como `var args` (número variable de argumentos), que es una nueva característica introducida en JDK 1.5. Acepta un array o una secuencia de argumentos separados por comas. El compilador empaqueta automáticamente los argumentos separados por comas en un array. Los tres puntos solo se pueden usar para el último argumento del método.

Pistas:

```
public class MyPolynomial {
    private double[] coeffs;
    public MyPolynomial(double... coeffs) { // varargs
        this.coeffs = coeffs; // varargs is treated as array
    }
    .....
}
```



```
}

// Test program
// Can invoke with a variable number of arguments
MyPolynomial p1 = new MyPolynomial(1.1, 2.2, 3.3);
MyPolynomial p1 = new MyPolynomial(1.1, 2.2, 3.3, 4.4, 5.5);
// Can also invoke with an array
Double coeffs = {1.2, 3.4, 5.6, 7.8}
MyPolynomial p2 = new MyPolynomial(coeffs);
```

- Otro constructor que toma coeficientes de un archivo (del nombre de archivo dado), que tiene este formato:

```
Degree-n(int)
c0(double)
c1(double)
.....
.....
cn-1(double)
cn(double)
(end-of-file)
```

Consejos:

```
public MyPolynomial(String filename) {
    Scanner in = null;
    try {
        in = new Scanner(new File(filename)); // open file
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    }
    int degree = in.nextInt(); // read the degree
    coeffs = new double[degree+1]; // allocate the array
    for (int i=0; i<coeffs.length; ++i) {
        coeffs[i] = in.nextDouble();
    }
}
```

- Un método `getDegree()` que devuelve el grado de este polinomio.
- Un método `toString()` que devuelve $cnx^n + cn-1x^{(n-1)} + \dots + c1x + c0$.
- Un método `evalute(double x)` que evalúan el polinomio para el x dado, sustituyendo la x en la expresión polinómica.
- Métodos `add()` y `multiply()` que suma y multiplica este polinomio con la instancia de `MyPolynomial` dada como parámetro (`another`) y devuelve una nueva instancia de `MyPolynomial` que contiene el resultado.

Escribe la clase `MyPolynomial`. También escribe un programa de prueba (llamado `TestMyPolynomial`) para probar todos los métodos definidos en la clase.

Pregunta: ¿Es necesario mantener el grado del polinomio como una variable de instancia en la clase `MyPolynomial` en Java?